T&G BnB

Technical document

How the direct-booking website for the T&G holiday homes in Kenya is built and hosted, how bookings and payments work, and how to run and extend it.

Website	**tg-bnb-lake.vercel.app**
API	**tg-bnb-api.onrender.com**
Describes	**The code as of 14 Nov 2026**
Written for	**The owner, and any developer who takes over the code**

Contents

1 Summary
2 How the system fits together
3 Who can do what
4 What guests can do
5 The owner's dashboard
6 Booking and payment
7 Cancellations, date changes and refunds
8 Emails and notifications
9 Security and privacy
10 Hosting and settings
11 Going live on the owner's accounts
12 Planned enhancements
13 Developer guide: backend
14 Developer guide: website (client)
15 Data model
16 API reference

1 Summary

T&G BnB is the owner's own booking website, not a marketplace. Guests browse the homes, see the full price for their dates, and pay online in Kenyan shillings through Paystack (M-Pesa or card). The owner and their team run everything from a dashboard: homes, calendar, bookings, refunds, guests, messages, reviews and money.

Kenya only
Every home has a Kenyan address. Prices in KES. Times in Nairobi time.

No fees
Price = nights × nightly rate + cleaning charge. Nothing added at checkout.

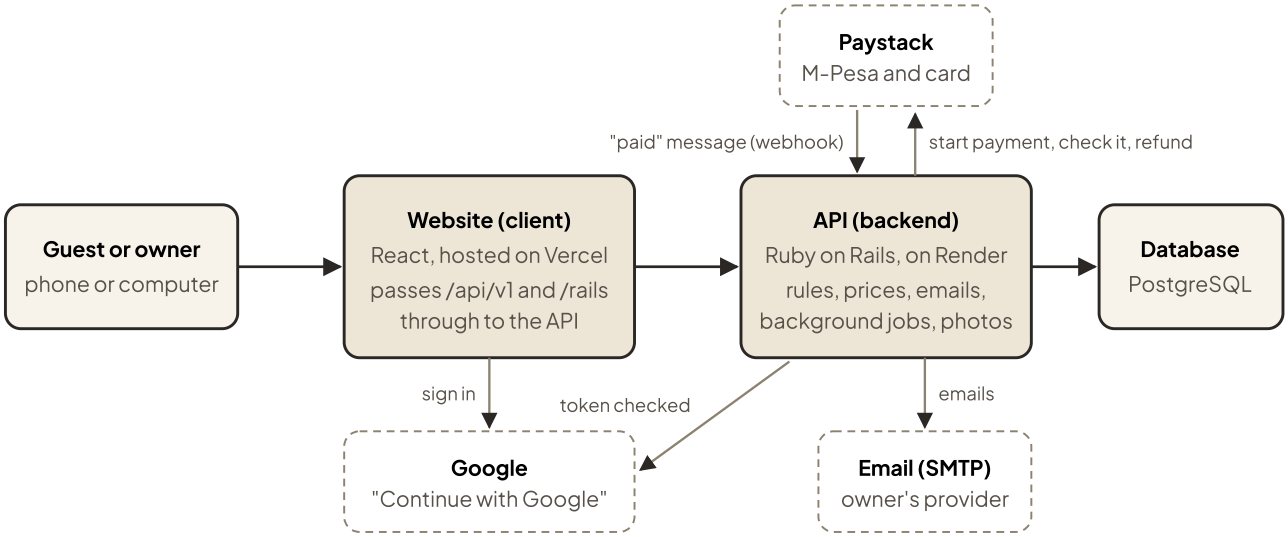
142 tests
Backend tests all pass, with lint and security scans clean.

Highlights

- **Server-side pricing.** Every price is worked out on the server from the home's settings, so a guest can never change what they pay.
- **No double bookings.** The database itself refuses overlapping stays, even when two guests pay at the same moment.
- **Local payments.** M-Pesa and card through Paystack's secure checkout, with refunds sent from the dashboard.
- **Instant booking or requests**, chosen per home, with Flexible, Moderate or Strict cancellation policies.
- **A complete owner dashboard**: homes, calendar, bookings, refunds, guests, inbox, reviews, finances, team and an activity log.
- **Built for phones first**, accessible by keyboard and screen reader, with the privacy features Kenya's Data Protection Act asks for (data download, account closure, consent).

2 How the system fits together

There are two programs: the website guests see (the client) and the server that holds the data and rules (the API). The browser only ever talks to one address, which keeps sign-in cookies simple and safe.



Part	What it does	Where it runs today
Website (client)	All pages guests and the owner see. Holds no data or rules of its own; it asks the API for everything.	Vercel project `tg-bnb`
API (backend)	Accounts, homes, prices, availability, bookings, payments, refunds, messages, reviews, emails, the owner's tools, and uploaded photos and videos.	Render web service `tg-bnb-api`, Frankfurt, free plan
Database	Everything the site stores. Also holds the background job queue and cache, so no extra service is needed.	Render Postgres `tg-bnb-db`, free plan
Paystack	Takes the guest's payment on its own secure page and sends refunds. The site never sees card or M-Pesa details.	Owner's Paystack account
Email	Sends verification links, password resets, team invites, receipts and booking updates through any SMTP provider.	Owner's email provider
Google sign-in	Optional "Continue with Google" for guests. The API checks every Google token with Google.	Owner's Google Cloud project

3 Who can do what

Who	How they get access	What they can do
Visitor	Not signed in	Home page, search, home pages with prices and availability, contact form, legal pages. Cannot book, save or message.
Guest	Signs up with email and password, or with Google. Every public sign-up is a guest.	Book and pay, manage their trips, change dates, cancel, save homes, message the owner, review stays, manage their profile and settings, download their data, close their account.
Team (owner and staff)	Added by an existing team member from **Team**. Nobody can sign up as team.	Everything in the owner's dashboard (section 5).

> **Team access**
>
> Team members have full access to the dashboard. Any member can invite or remove others (but not themselves). A removed member's account becomes a guest account and is signed out everywhere. Every team action is recorded in the Activity log.

4 What guests can do

Feature	How it works
Find a home	Search by place, dates, number of guests, price, category and amenities, with sorting. Only homes free for the chosen dates are shown. The search lives in the address bar, so a search can be shared as a link.
Home page	Photos and videos, description, amenities, house rules, check-in and check-out times, cancellation policy, licence number, reviews, a calendar of free nights, and the exact total for the chosen dates.
Book and pay	Sign in, then pay on Paystack's page by M-Pesa or card. Section 6 has the details.
Trips	Every booking with its status, receipt, check-in details (only once confirmed), and options to change dates, cancel or message the owner.
Saved homes	A heart on any home adds it to the guest's saved list.
Messages	Conversations with the owner, about a booking or a home.
Reviews	After check-out, a 1 to 5 star rating and comment. The owner can reply or hide abusive reviews.
Notifications	A bell with unread updates, refreshed every minute while the page is open, plus emails (section 8).
Account	Name, phone, photo, password, email verification, marketing consent, message emails on or off, data download (JSON), and account closure.

5 The owner's dashboard

Signed-in team members see these pages under `/account`. Guests who open these addresses are turned away, and the API refuses them too, whatever the browser sends.

Page	What it is for
Overview	Today's arrivals and departures, guests staying now, requests and refunds waiting, unread messages, occupancy, this year's money, and recent reviews.
Homes	Create, edit, publish, unlist or delete homes. A home with past bookings is archived rather than deleted, so its records stay. Set the following: photos (up to 20 per upload, 10 MB each, resized to at most 2000 px) and videos (MP4, WebM or MOV, 200 MB each), or links to them; price, cleaning charge and guest limit; bedrooms, beds and bathrooms; check-in and check-out times; minimum and maximum nights; instant booking or requests; cancellation policy; house rules; private check-in instructions and wifi details; licence number; category; amenities. Owners can add their own categories and amenities.
Calendar	A month view across all homes. Block nights (repairs, own use) with a reason, or reopen them. Blocked nights disappear from booking straight away. Nights a guest has booked cannot be blocked.
Bookings	Every booking, with filters. Accept or decline requests, cancel, change dates, and settle refunds (section 7).
Guests	Everyone who has booked, with their number of stays, total spent and latest stay.
Inbox	Guest conversations and contact-form messages. Messages can be marked as dealt with.
Reviews	Reply to reviews, or hide one. A hidden review leaves the public page and the home's rating.
Finances	For any year: money received, refunded and net, by month and by home (stays, nights, received, refunds, net); refunds still owed; stays booked ahead; every payment, with CSV download.
Team	Add a team member by email. They get an invite link, valid for 7 days, to choose a password.
Activity	A log of every team action: who did what, to what, and when.

6 Booking and payment

The server works out every price from the home's own settings. The browser sends only the home, the dates and the number of guests, so a guest cannot change what they pay.

Steps

1. The guest picks dates and guests. The server checks the dates against the home's rules: not in the past, within the minimum and maximum nights, within the guest limit, the home published, and no booked or blocked nights in the way.
2. The server creates the booking as **awaiting payment** and holds the dates for **15 minutes**. It saves the price at that moment: nightly rate, nights, cleaning charge and total.
3. The guest goes to Paystack's checkout page and pays by M-Pesa or card.
4. Paystack tells the server the payment succeeded, in two ways: the guest's browser returns to the site and the server asks Paystack to confirm it, and Paystack also sends a signed "paid" message (webhook) even if the guest closes the page. Either way the result is the same, and a payment is only ever counted once. The server checks that the amount and currency match what was charged.
5. **Instant booking** homes confirm straight away and the guest gets a receipt. **Request** homes become a request; the owner has **24 hours** to accept or decline. A request that gets no answer is declined automatically and refunded in full.

What stops double bookings

The database itself refuses two active bookings (awaiting payment, requested or confirmed) on the same home with overlapping nights. Even if two people press "Pay" at the same moment, only one gets the nights. The database also checks that the total always equals nights × nightly rate + cleaning charge.

Unpaid holds and late payments

- A background task runs every minute. It frees holds that weren't paid within 15 minutes and declines requests that are past 24 hours.
- If a payment arrives after its hold expired and the nights are still free, the booking still goes through. If someone else has taken the nights by then, the payment is marked as a refund due, and the owner sees it in Bookings and Finances.

Booking statuses

Status	Meaning
Awaiting payment	Dates held for 15 minutes while the guest pays.
Requested	Paid, waiting for the owner to accept (request homes only).
Confirmed	Paid and confirmed. The guest can see the check-in instructions and wifi details.
Declined	The owner declined the request, or did not answer in 24 hours. Full refund due.
Expired	Not paid in time. The dates are free again.
Cancelled by guest / by owner	Cancelled. The refund depends on who cancelled (section 7).

7 Cancellations, date changes and refunds

Cancellation policies (chosen per home)

Policy	Guest gets back
Flexible	Everything, if they cancel before the day of check-in.
Moderate	Everything at least 5 days before check-in; half after that.
Strict	Half at least 7 days before check-in; nothing after that.

- Guests can cancel until the day before check-in. They see the refund before they confirm.
- A request the owner hasn't accepted yet is always refunded in full if the guest cancels it.
- When the owner cancels a stay or declines a request, the guest gets everything back.

Changing dates

Guests (before check-in) and the owner (until check-out) can move a confirmed or requested stay to other free dates. The nightly price stays the same. A shorter stay refunds the difference. A longer stay is refused; the guest books the extra nights separately.

How a refund is paid

Cancelling, declining or shortening a stay works out the refund and marks it as **due** on the payment. The owner then settles it on the booking's page in one of two ways:

- **Refund through Paystack**: the site asks Paystack to send the money back to the guest's card or M-Pesa.
- **I refunded it in Paystack**: for a refund the owner already sent from the Paystack dashboard; the site only records it.

Either way the guest is told, the Finances page updates, and the action is logged.

8 Emails and notifications

Each update creates an item in the person's notification bell and, where listed, an email. Emails are sent in the background so pages stay fast.

Email	To	When
Confirm your email	New account	After sign-up. Link valid for 3 days.
Reset your password	Anyone	"Forgot password". Link valid for 15 minutes and works once. Resetting signs the person out everywhere else.
Team invitation	New team member	When added on the Team page. Link valid for 7 days.
Booking receipt	Guest	When a stay is confirmed.
New booking / booking request	Every team member	When a stay is confirmed, or a request is paid.
Request sent, declined	Guest	When a request is paid, or declined.
Booking cancelled	Guest, and the team if the guest cancelled	On any cancellation, with the refund amount.
Dates changed	The other side	When the guest or the owner moves a stay.
Refund sent	Guest	When a refund is recorded or sent through Paystack.
New message	Other side	On every message, unless the person turned message emails off in Settings.
Contact form	Every team member	When someone uses the contact form.

New reviews appear only in the bell, with no email.

> **Email delivery**
> Emails are sent through the SMTP provider set in the server settings (section 10). On a developer's machine they are saved as files in `backend/tmp/mails` instead of being sent.

9 Security and privacy

- **Prices and rules on the server.** The browser cannot set a price, a refund, a booking status or a role.
- **Access checked on every request.** Every owner action checks that the signed-in person is on the team. Guests only ever see their own bookings, messages and data.
- **Sign-in.** Passwords are at least 8 characters and stored only as bcrypt hashes. The session cookie is signed, hidden from page scripts (httpOnly), HTTPS-only in production, and SameSite=Lax. The API also

refuses changes sent from other websites (an Origin check), which blocks cross-site form tricks. The live site forces HTTPS.

- **Payments.** Card and M-Pesa details never reach the site; Paystack handles them. Paystack's "paid" messages are accepted only with a valid signature (HMAC-SHA512 with the secret key), and amounts are checked against the booking.
- **Limits on repeated attempts** from the same IP address, for example: sign-in and sign-up 10 per 3 minutes, password reset emails 5 per 10 minutes, verification emails 3 per 10 minutes, bookings 20 per 10 minutes, messages 30 per 10 minutes, contact form 5 per 10 minutes.
- **Uploads** are team-only and limited by type and size. Photos are re-encoded and resized on upload.
- **Private details**: a home's check-in instructions and wifi details are shown only to a guest with a confirmed stay.
- **Audit log** of every team action, with the person and IP address.
- **Data protection (Kenya, Data Protection Act 2019).** There are privacy and terms pages, and a cookie notice. Marketing consent is opt-in and recorded with a date. Guests can download all their data. Closing an account removes the person's name, email, phone, photo and sign-in, but keeps their bookings and payments for the owner's tax and dispute records.
- **Checks on every change**: a static security scan (Brakeman) and a check of known-vulnerable libraries (bundler-audit) run in GitHub Actions with the tests. Both are clean.

10 Hosting and settings

How a new version goes live

- **API:** Render rebuilds from `backend/Dockerfile` on every push to the `main` branch that changes `backend/`. On start it runs any database updates (migrations). The first start on an empty database loads the schema and the demo homes.
- **Website:** deployed with `vercel deploy --prod` from `client/`. The Vercel project can also be connected to the Git repository to deploy on every push. `client/vercel.json` forwards `/api/v1` and `/rails` to the Render API. If the API address changes, update that file.
- **Background jobs** (emails, expiring holds) run inside the API process (`SOLID_QUEUE_IN_PUMA=1`), so no separate worker is needed.

API settings (environment variables on Render)

Setting	Purpose	Needed for
`DATABASE_URL`	Database connection.	Always
`RAILS_MASTER_KEY`	Unlocks the app's encrypted secrets (signs cookies). Keep it private; losing it signs everyone out.	Always
`APP_URL`	The website address, used in email links and Paystack's return link.	Always
`SOLID_QUEUE_IN_PUMA`	`1` runs background jobs inside the API.	Always
`PAYSTACK_SECRET_KEY`	Paystack secret key: `sk_test_…` for testing, `sk_live_…` for real payments. Find it under Settings, API Keys & Webhooks in Paystack.	Payments
`SMTP_ADDRESS`, `SMTP_PORT`, `SMTP_USERNAME`, `SMTP_PASSWORD`, `MAIL_FROM`	Any SMTP provider (Gmail, Zoho, Brevo, SendGrid and others). The port defaults to 587. Use an address on the owner's domain for `MAIL_FROM`.	Emails
`GOOGLE_CLIENT_ID`	Google OAuth client ID, the same one as the website's.	Google sign-in
`SEED_ADMIN_EMAIL`, `SEED_ADMIN_PASSWORD`	Create the first team account when seeding a production database. No admin is created without both.	First setup
`CORS_ORIGINS`	Only if the website is served from a different address than the API. Not needed with the Vercel forwarding.	Not needed

Website settings (on Vercel, at build time)

Setting	Purpose	Needed for
`VITE_GOOGLE_CLIENT_ID`	Shows the "Continue with Google" button. Hidden when not set.	Google sign-in

Setting	Purpose	Needed for
`VITE_API_URL`	Only to call an API on another address. Leave unset with the Vercel forwarding.	**Not needed**

Paystack dashboard

Set the webhook URL to `https://<website address>/api/v1/webhooks/paystack`. For the current site: `https://tg-bnb-lake.vercel.app/api/v1/webhooks/paystack`. Enable M-Pesa and card for KES.

11 Going live on the owner's accounts

The system is complete. Going live means connecting it to the owner's own accounts and production hosting. Each step is configuration only; no code changes are needed.

#	Step	What to do
1	Production hosting	Move the database to a paid plan with daily backups, and the API to an always-on plan so pages answer instantly at any hour. The same Docker image also runs on the owner's own server.
2	File storage	Attach a persistent disk or S3-compatible storage, so photos and videos the owner uploads are kept across updates.
3	Paystack	Set `PAYSTACK_SECRET_KEY` and the webhook URL (section 10). Make one booking and one refund with the test key, then switch to the live key.
4	Email	Set the SMTP settings and `MAIL_FROM`, then send a test sign-up, password reset and team invite.
5	Homes and team	Create the owner's team account, invite staff from Team, and replace the 12 demo homes with the real ones.
6	Domain	Point the owner's domain at the website, then update `APP_URL` and the Paystack webhook URL to match.
7	Google sign-in (optional)	Create an OAuth client in Google Cloud with the website address as an authorised origin. Set `GOOGLE_CLIENT_ID` and `VITE_GOOGLE_CLIENT_ID`, then redeploy the website.
8	Tax and registration	Confirm VAT on receipts, the tourism levy and short-stay registration with the owner's adviser, and review the privacy and terms wording.

12 Planned enhancements

The system is built so these can be added without reworking what exists.

- **Seasonal and weekend pricing**, length-of-stay discounts and extra-guest charges.
- **Calendar sync** (iCal) with Airbnb and Booking.com, so nights booked elsewhere close automatically.
- **Team roles**, for example staff who manage bookings but not finances.
- **Extending a stay** from the booking page, charging only the extra nights.
- **Automated website tests** (Vitest and Playwright), alongside the existing backend tests.

13 Developer guide: backend

Rails 8.1 in API mode, Ruby 4.0.2, PostgreSQL (UUID keys; `pgcrypto`, `citext`, `btree_gist`). JSON under `/api/v1`. Uploaded files are served under `/rails` (Active Storage). Time zone: Nairobi.

Run it

```
cd backend
bin/setup --skip-server     # gems, databases, migrations
bin/rails db:seed           # amenities, 12 Kenyan demo homes, dev accounts
bin/rails server            # http://localhost:3000, health check GET /up
```

In development the seed creates a team account `admin@tgbnb.test` and a guest `guest@tgbnb.test`, with the passwords in `db/seeds.rb`. These work on a developer's machine only.

Checks (all must pass; CI runs them on every change to `backend/`)

```
bin/rails test              # Minitest, 142 tests
bin/rubocop                 # style
bin/brakeman --no-pager     # security scan
bin/bundler-audit           # vulnerable gems
```

Tests use a fake Paystack client and a fake Google check, so they never call outside services.

Where things live

Path	Contents
`app/controllers/api/v1/`	Guest and public endpoints. `admin/` holds team endpoints, each with `before_action :require_admin`. `webhooks/paystack_controller.rb` handles Paystack.
`app/controllers/concerns/`	`authentication.rb` (cookie sessions), `json_renderers.rb` (shared JSON shapes), `paystack_checkout.rb` (starting a payment).
`app/models/booking.rb`	The booking rules: holds, confirm after payment, accept, decline, cancel, refund amounts, date changes.
`app/models/payment.rb`	Settling a Paystack transaction (once only), flagging and sending refunds.
`app/models/paystack.rb`	A small Paystack API client: initialise, verify, refund, webhook signature.
`app/models/notify.rb`	Every notification and the emails that go with it.
`app/models/google_identity.rb`	Checks Google ID tokens with Google.
`app/mailers/`	Account, password, booking receipt and notice emails.

Path	Contents
`config/recurring.yml`	Scheduled jobs (production only): expire holds every minute; clear finished jobs hourly.
`db/seeds.rb`, `db/seeds/listings.json`	Amenities, demo homes, dev accounts.

Conventions

- Money is whole integers in cents (`*_cents`), always KES. Bookings save their price at the time of booking.
- Errors are always `{ "error": { "code", "message", "details"? } }`. Validation errors use code `validation_failed`, with field messages in `details`.
- Lists page with `?page=`; responses include the next page number when there is one.
- Team actions call `audit(action, target, summary)` to write the Activity log.
- Key rules are also enforced in the database: no overlapping active bookings, totals that match, dates in order, valid statuses.
- In development, expired holds are not cleared automatically (the schedule runs in production only). Run `Booking.expire_stale_holds!` in the console if needed.

14 Developer guide: website (client)

React 18 with Vite 4, Tailwind CSS 3, React Router 6, axios, Radix UI (dialog, popover, avatar) in a shadcn-style component folder, react-day-picker 8, date-fns 2, lucide-react icons, react-toastify. Plain JavaScript (JSX), no TypeScript.

Run it

```
cd client
yarn install
yarn dev        # http://localhost:5173; forwards /api/v1 and /rails to RAILS_API_URL (default
http://localhost:3000)
yarn build      # production files in dist/
```

Structure

Path	Contents
`src/App.jsx`	All routes, the app's providers, toasts and cookie notice.
`src/pages/`	One file per page. `home/` holds the landing page sections.
`src/components/ui/`	Shared components: layouts (`Layout`, `DashboardLayout`, `AuthLayout`), route guards (`RequireSignIn`, `RequireOwner`), dialogs, booking widget, date picker, and the shadcn-style primitives.

Path	Contents
`src/providers/`	Shared state: the signed-in user, homes, saved homes, notifications (refreshed every 60 s while the tab is visible).
`hooks/index.js`	`useAuth` and other hooks over the providers.
`src/utils/axios.js`	The single API client (`withCredentials` for the session cookie), plus `errorMessageFrom` and `fieldErrorsFrom` to show server errors.
`src/utils/`	Money and date formatting, booking status labels, search filters, recently viewed.
`tailwind.config.cjs`	Colours `sand` #EDE6D6, `ink` #2A2725, `paper` #F7F3EA, and warm greys. Font: Plus Jakarta Sans.

Routes

Address	Page
`/`	Home page
`/homes` (`/stays` redirects)	Search results
`/place/:id`	A home
`/login`, `/register`, `/forgot-password`, `/reset-password`, `/verify-email`	Accounts. `/reset-password?welcome=1` is also the team invite page.
`/privacy`, `/terms`, `/contact`	Legal and contact pages
`/account`, `/account/profile`, `/account/settings`, `/account/notifications`	Signed in: overview (guest or owner view), profile, settings, notifications
`/account/bookings[/:id[/receipt]]`, `/account/saved`, `/account/messages[/:id]`	Signed in: trips, receipt, saved homes, messages
`/account/places[/new|/:id]`, `/account/reservations[/:id]`, `/account/calendar`, `/account/inbox[/:id]`, `/account/guests[/:id]`, `/account/reviews`, `/account/finances`, `/account/team`, `/account/activity`	Team only (`RequireOwner`): homes, bookings, calendar, inbox, guests, reviews, finances, team, activity

Things to know

- Use explicit `h-* w-*` rather than `size-*` on components that already set a height or width: the installed tailwind-merge (1.14) does not recognise `size-*` as conflicting.

- React 18.2 needs the lowercase `fetchpriority` attribute on images.
- Every page must work at 360 px wide, by keyboard, with visible focus.
- `netlify.toml` is left over from an earlier host and is not used.

15 Data model

Every table uses UUID keys and timestamps. Money is in integer cents (KES).

Table	Holds
users	Email (unique, case-insensitive), name, password hash, role (`guest` or `admin`), phone, email verified at, marketing consent at, message emails on/off, Google ID, closed at. Avatar photo via Active Storage.
sessions	One row per signed-in browser: user, IP address, user agent.
listings	A home: title, slug, description, type, address (always Kenya), guests, bedrooms, beds, bathrooms, nightly price, cleaning charge, times, min/max nights, booking type, cancellation policy, house rules, check-in instructions, wifi details, licence number, status (draft, published, unlisted), featured, category, cached rating and review count, deleted at.
listing_photos, listing_videos	Ordered photo and video URLs (uploaded files or outside links).
categories, amenities, listing_amenities	Owner-defined categories, and amenities linked to homes.
availability_blocks	Nights the owner closed: home, from, to (exclusive), reason.
bookings	Home, guest, check-in, check-out, guests, nights, saved nightly price, cleaning charge and total, status, request deadline, cancelled at and reason. Database rules: no overlap for active bookings, total = nights × nightly + cleaning.
payments	Booking, Paystack reference (unique), amount, currency, status (pending, succeeded, failed), channel, Paystack transaction ID, paid at; refund due, refund amount, refunded at, Paystack refund ID.
reviews	One per booking: rating 1 to 5, comment, owner reply, hidden.
saved_listings	A guest's saved homes.
conversations, messages	A guest's thread with the owner, about a booking or a home, with read markers for each side; the messages in it.
notifications	Bell items: kind, title, text, link, read at.
contact_messages	Contact form entries and when they were dealt with.
audit_logs	Team actions: who, action, target, summary, IP address.

Table	Holds
solid_queue_*, solid_cache_*	Background job queue and cache, kept in the same database.

16 API reference

All under `/api/v1`, JSON in and out. Signed-in requests use the `session_id` cookie. Team endpoints return 403 for guests and 401 for visitors.

Endpoint	Purpose
`POST /registrations`	Sign up (guest)
`GET | POST | DELETE /session`	Current user, sign in, sign out
`POST /google_session`	Sign in with a Google ID token
`PATCH | DELETE /profile`, `GET /profile/export`	Update profile and settings, close account, download data
`POST | PATCH /email_verification`	Send a verification link, confirm it
`POST | PATCH /password_reset`	Send a reset link, set a new password (also accepts invite tokens)
`POST /contact_messages`	Contact form
`GET /listings`, `GET /listings/:id`	Search homes, one home (with quote and unavailable nights)
`GET | POST | DELETE /saved_listings[/:listing_id]`	Saved homes
`GET | POST /bookings`, `GET /bookings/:id`	A guest's trips, book
`POST /bookings/:id/cancel | reschedule`	Guest cancels or changes dates
`POST /bookings/:id/payment`, `POST /bookings/:id/payment/verify`	Start Paystack checkout; check the payment after returning
`POST /bookings/:id/review`	Review a past stay
`GET | POST /conversations[/:id]`, `POST /conversations/:id/messages`	Guest messaging
`GET /notifications`, `POST /notifications/:id/read | read_all`	Notification bell
`POST /webhooks/paystack`	Paystack "paid" messages (signed)
Team (`/api/v1/admin`)	

Endpoint	Purpose
`GET /overview`, `GET /calendar`, `GET /finances`	Dashboard, month calendar, money for a year
`/listings` (list, show, create, update, delete)	Homes (delete archives a home with bookings)
`POST /listings/:id/blocks`, `DELETE /blocks/:id`	Block and reopen nights
`/categories`, `/amenities`	Owner's categories and amenities
`POST /uploads`	Upload photos or videos
`GET /bookings[/:id]`	All bookings, one booking
`POST /bookings/:id/accept | decline | cancel | reschedule | refund | mark_refunded`	Booking actions and refunds
`GET /guests[/:id]`	Guests and their history
`/conversations`, `/conversations/:id/messages`	Inbox
`GET /contact_messages`, `POST /contact_messages/:id/handle`	Contact form messages
`GET /reviews`, `PATCH /reviews/:id`	Reply, hide or show
`GET | POST /staff`, `DELETE /staff/:id`	Team members
`GET /audit_logs`	Activity log

Other

`GET /up`	Health check: 200 when the API is running